

Approches classiques de l'IA

Chapitre 6 : Agents logiques

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 6. Agents logiques

- 1 Agents basés sur la connaissance
- 2 Le monde de Wumpus
- 3 Logique (en général) — modèles et implications
- 4 Logique propositionnelle
- 5 Règles d'inférence et preuves de théorèmes
- 6 Résumé du chapitre

Agents basés sur la connaissance

Agents basés sur la connaissance

Chapitres 3 à 5 (résolution de problèmes par exploration) :

- représentation **atomique** (états) $\Rightarrow$ limitations (en particulier dans env. partiellement observable)
- **connaissance très spécifique, limitée et fixée** (définie implicitement par le modèle de transition — fonction RESULT)

Agents basés sur la connaissance

Chapitres 3 à 5 (résolution de problèmes par exploration) :

- représentation **atomique** (états) $\Rightarrow$ limitations (en particulier dans env. partiellement observable)
- **connaissance très spécifique, limitée et fixée** (définie implicitement par le modèle de transition — fonction RESULT)

Dans ce chapitre :

- agents basés sur la **connaissance**, représentée de manière beaucoup plus flexible et extensible, grâce à la **logique**
- approche d'IA qui va permettre d'**inférer** et de **raisonner**

Agents basés sur la connaissance

Agents dotés de raisonnement ?

Agents basés sur la connaissance

Agents dotés de raisonnement ?

Intuition : utilisation de la **logique**

- **représentations** d'un monde complexe
- processus d'inférence pour dériver de nouvelles **représentations**
- utilisation des nouvelles représentations pour **déduire ce qu'il faut faire**

Agents basés sur la connaissance

Agents dotés de raisonnement ?

Intuition : utilisation de la **logique**

- **représentations** d'un monde complexe
- processus d'inférence pour dériver de nouvelles **représentations**
- utilisation des nouvelles représentations pour **déduire ce qu'il faut faire**

représentations $\simeq$ connaissance

Agents basés sur la connaissance

L'élément central d'un agent basé sur la connaissance est sa **base de connaissances** (KB — Knowledge Base).

KB = ensemble de **phrases** dans un langage **formel** (appelé le **langage de représentation de la connaissance**).

Agents basés sur la connaissance

Opérations de base :

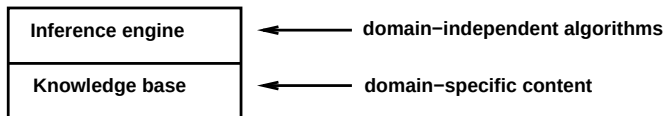
- TELL : permet d'ajouter de nouvelles phrases à la KB (“Tell it what it needs to know”) ;
- ASK : permet de savoir ce qui est connu (“Then it can Ask itself what to do”).

Agents basés sur la connaissance

Opérations de base :

- TELL : permet d'ajouter de nouvelles phrases à la KB (“Tell it what it needs to know”) ;
- ASK : permet de savoir ce qui est connu (“Then it can Ask itself what to do”).

⇒ ces 2 opérations peuvent permettre l'inférence : dériver de nouvelles phrases à partir d'anciennes.



Agents basés sur la connaissance

Axiome = phrase qui n'est pas dérivée d'autres phrases.

Les axiomes forment la **connaissance initiale** (background knowledge).

La fonction suivante définit un agent simple basé sur la connaissance.

```

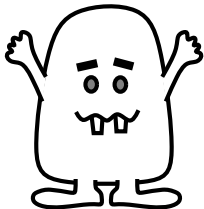
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
               t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
  
```

Le monde de Wumpus

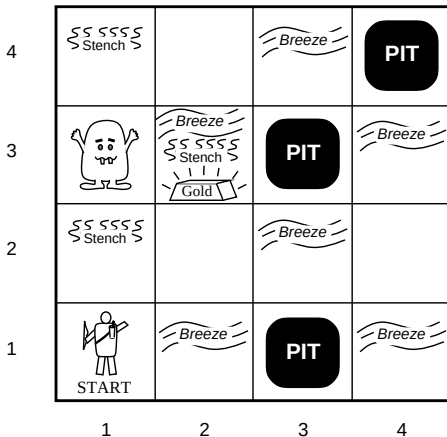
Chasse au Wumpus

Le jeu **Wumpus Hunt** est un des plus anciens jeux informatiques (écrit en BASIC (1976) et joué en ligne de commande).



- Le **monde de Wumpus** est une cave constituée de pièces.
- Dans une de ces pièces se trouve l'horrible Wumpus, une bête qui dévore quiconque pénètre dans son antre.
- Wumpus peut être tué par un agent, mais celui-ci n'a qu'une seule flèche.
- Certaines pièces contiennent des trous profonds (**pit**) qui piègent quiconque s'y aventure.
- Une pièce contient un énorme tas d'or.

Le monde de Wumpus : exemple



Défaite : mort ; **Victoire** : sortir de la cave avec de l'or.

Perception : on ne voit pas ce qui se trouve dans une pièce adjacente (mais on perçoit certaines choses comme des odeurs).

Description du monde de Wumpus

- **Mesure de Performance** : +1000 pour sortir de la cave avec l'or ; -1000 en cas de mort (tombé dans un trou ou dévoré) ; -1 par action réalisée et -10 pour utiliser l'unique flèche
- **Environnement** : grille 4×4 de pièces dont 3 sont spéciales (départ, or et Wumpus) :
 - ▶ L'agent démarre toujours de la case [1,1], orienté vers la droite.
 - ▶ Localisations de l'or et du Wumpus : choisies aléatoirement (distribution uniforme parmi les carrés $\neq [1, 1]$).
 - ▶ chaque **autre** carré : 20 % de chance d'être un trou.

Description du monde de Wumpus

- **Actuateurs** : 6 actions possibles

- ▶ tourner à gauche de 90° [*Left*]; tourner à droite de 90° [*Right*] : change l'orientation de l'agent (mais pas la case courante) ;
- ▶ avancer tout droit [*Forward*] : mort de l'agent si Wumpus vivant ou trou (case destination) ; pas de déplacement si mur ;
- ▶ prendre l'or s'il y en a sur la case courante [*Grab*] ;
- ▶ tirer une flèche [*Shoot*] (ligne droite dans la direction de l'agent) : la flèche continue jusqu'à ce qu'elle rencontre un mur ou Wumpus ; seule la première action [*Shoot*] a un effet ;
- ▶ sortir de la cave [*Climb*] (uniquement depuis la case $[1, 1]$).

Description du monde de Wumpus

- **Senseurs** : 5 perceptions possibles :
 - ▶ *[Stench]* (puanteur) : si le carré contenant le Wumpus est directement adjacent (pas en diagonale) ;
 - ▶ *[Breeze]* (courant d'air) : si un carré contenant un trou est directement adjacent (pas en diagonale) ;
 - ▶ *[Glitter]* (scintillement) : si le carré courant contient l'or ;
 - ▶ *[Bump]* (choc) : quand l'agent avance droit dans un mur ;
 - ▶ *[Scream]* (hurlement) : quand le Wumpus est tué (perçu dans toute la cave).

Caractérisation du monde de Wumpus

Observable ?? Non — uniquement perception *locale*

Déterministe ?? Oui — le hasard n'intervient pas (une fois la carte créée)

Statique ?? Oui — Wumpus et les trous ne bougent pas

Discret ?? Oui

Agent unique ?? Oui — Wumpus est une “caractéristique naturelle” qui ne bouge pas

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 	2,1	3,1	4,1

Légende :

- A = Agent

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

Légende :

- A = Agent
- OK = carré sûr

Perception : [Stench, Breeze, Glitter, Bump, Scream]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

Légende :

- A = Agent
- OK = carré sûr

Perception : [Stench, Breeze, Glitter, Bump, Scream]

Action choisie : [Forward]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 B A OK	3,1 P?	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*

Perception : [Stench, Breeze, Glitter, Bump, Scream]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 B  OK	3,1 P?	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*

Perception : [~~Stench~~, **Breeze**, ~~Glitter~~, ~~Bump~~, ~~Scream~~]

Actions choisies : [*Left*, *Left*, *Forward*, *Right*, *Forward*]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2  S OK P? W?	2,2	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Perception : [**Stench**, **Breeze**, **Glitter**, **Bump**, **Scream**]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2 S  OK P W?	2,2	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Déductions : [2,2] ne peut pas être P car pas de B en [1,2]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2 S  OK P W?	2,2	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Déductions : [2,2] ne peut pas être P car pas de B en [1,2]
 $\Rightarrow$ [3,1] est P

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2 S  OK P W	2,2	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Déductions : [2,2] ne peut pas être W car pas de S en [2,1]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 S  OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Déductions : [2,2] ne peut pas être W car pas de S en [2,1]
 $\Rightarrow$ [2,2] est OK et [1,3] est W

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 S  OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Actions choisies : *[Right, Forward]*

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 S V OK	2,2  OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Perception : [Stench, Breeze, Glitter, Bump, Scream]

Explorer le monde de Wumpus (exemple)

1,4	2,4	3,4	4,4
1,3 W!	2,3 OK	3,3	4,3
1,2 S V OK	2,2  OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*

Perception : [Stench, Breeze, Glitter, Bump, Scream]

Actions choisies : [Left, Forward]

Explorer le monde de Wumpus (exemple)

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 S,B,G  G!	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*
- G = *Glitter / Gold*

Perception : [Stench, Breeze, Glitter, ~~Bump~~, ~~Scream~~]

Explorer le monde de Wumpus (exemple)

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 S,B,G  G!	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

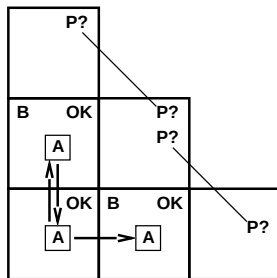
Légende :

- A = Agent
- OK = carré sûr
- V = visité
- B = *Breeze*
- P = *Pit*
- S = *Stench*
- W = *Wumpus*
- G = *Glitter / Gold*

Perception : [**Stench**, **Breeze**, **Glitter**, ~~Bump~~, ~~Scream~~]

Actions : prendre l'or et sortir via des carrés visités

Autres exemples : cas litigieux

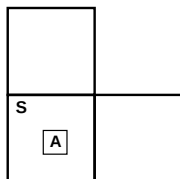


[Breeze] en [1,2] et [2,1]

⇒ pas d'action sûre

Mais [2,2] a une plus forte probabilité d'être un trou que les deux autres "P?".

Autres exemples : cas litigieux



[Stench] en [1,1]

⇒ bloqué

Mais peut utiliser la stratégie suivante :

- Action *Shoot* droit devant
- Wumpus était là ⇒ mort ⇒ sûr
- Wumpus n'était pas là ⇒ sûr

Logique (en général) — modèles et implications

Logique en général

Différents types de logiques : propositionnelle, de premier ordre, etc.

Logiques : langages formels permettant de représenter de l'information et d'en tirer des conclusions.

Syntaxe : définit les phrases admises par le langage.

Sémantique : définit la “signification” des phrases ; i.e., définit la **vérité** d'une phrase dans un “monde” donné.

Modèles

Un **modèle** est une structure permettant d'évaluer la vérité d'une phrase.

De plus, si une phrase α est vraie pour un modèle m , on dit que le modèle m **satisfait** α ou que m **est modèle de** α .

$M(\alpha)$ est l'ensemble de tous les modèles de α .

Exemple

Soit un environnement décrit par 2 variables A et B en logique propositionnelle (LP), et soit une phrase $\beta \equiv A \vee B$. Il y a 4 modèles possibles (valeurs booléennes pour (A, B)) : $(Vrai, Vrai)$, $(Vrai, Faux)$, $(Faux, Vrai)$ et $(Faux, Faux)$. Les trois premiers modèles satisfont β et forment l'ensemble $M(\beta)$.

Implication logique

Implication logique : α implique β (noté $\alpha \models \beta$) si et seulement si, pour chaque modèle pour lequel α est vrai, β est également vrai, i.e.,

$$\alpha \models \beta \iff M(\alpha) \subseteq M(\beta).$$

Exemple (suite)

Soit le même environnement que précédemment. Si $\alpha \equiv A$, alors $\alpha \models \beta$ car $M(\alpha) = \{(Vrai, Vrai), (Vrai, Faux)\} \subset M(\beta)$. L'inverse n'est pas vrai (le modèle $(Faux, Vrai)$ est dans $M(\beta)$ mais pas dans $M(\alpha)$).

Implication logique

Pour nos agents, la base de connaissances (KB , ensemble de phrases) peut impliquer de nouvelles connaissances (phrases).

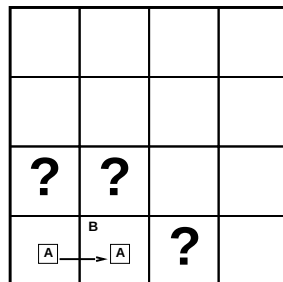
$$\begin{aligned}
 KB \models \alpha &\iff \alpha \text{ est vrai dans tous les modèles où } KB \text{ est vrai} \\
 &\iff M(KB) \subseteq M(\alpha)
 \end{aligned}$$

Note : KB peut être vu comme un ensemble de phrases ou comme une phrase unique qui vérifie toutes les phrases individuelles, d'où l'abus de notation pour $M(KB)$

Implication logique dans le monde de Wumpus

Situation après

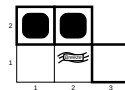
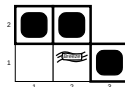
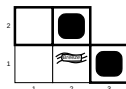
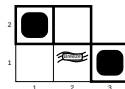
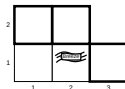
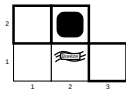
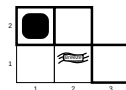
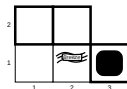
- n'avoir rien détecté en [1,1]
- avoir effectué un déplacement en [2,1]
- avoir perçu [*Breeze*] en [2, 1]



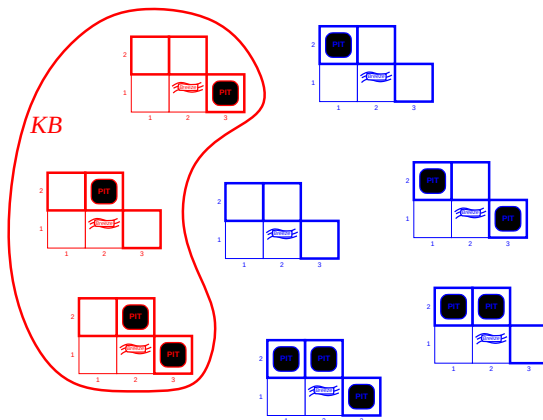
Si on se limite aux trous, quels sont les modèles possibles pour les cases “?” ?

3 choix booléens $\Rightarrow$ 8 modèles possibles

Modèles et Implications — Wumpus

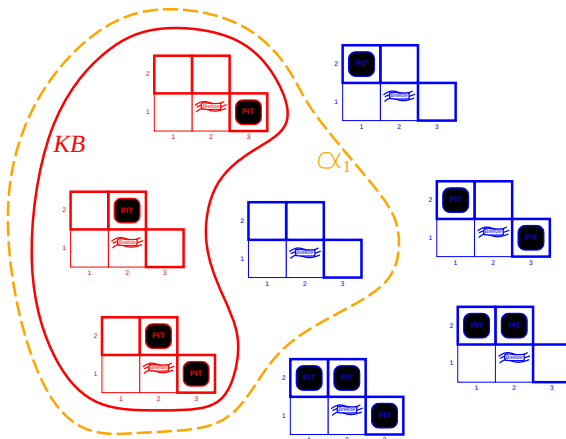


Modèles et Implications — Wumpus



KB = règles du monde de Wumpus + observations

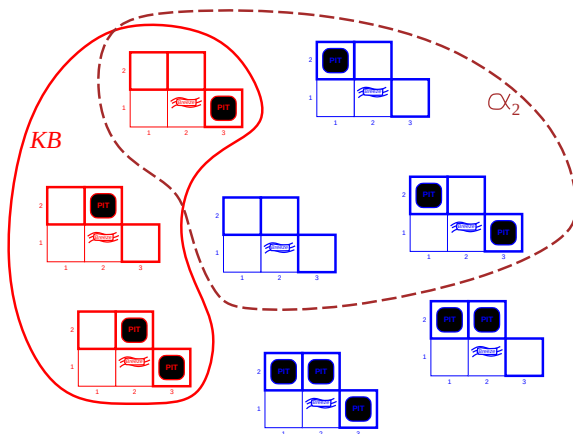
Modèles et Implications — Wumpus



KB = règles du monde de Wumpus + observations

α_1 = "[1,2] est sûr", $KB \models \alpha_1$, prouvé par **model checking**

Modèles et Implications — Wumpus



KB = règles du monde de Wumpus + observations

α_2 = "[2,2] est sûr", $KB \not\models \alpha_2$

Inférence logique

Dans l'exemple précédent : **inférence** par “model checking” (vérification de modèle), c'est-à-dire le fait de **dériver des conclusions** en appliquant la définition de l'implication logique ($\models$).

Inférence logique

Dans l'exemple précédent : **inférence** par “model checking” (vérification de modèle), c'est-à-dire le fait de **dériver des conclusions** en appliquant la définition de l'implication logique ($\models$).

$\Rightarrow$ divers algorithmes d'inférences possibles.

Notation : $KB \vdash_i \alpha$ = phrase α peut être dérivée de KB par la procédure i

Inférence logique

Un algorithme d'inférence est dit

- **sensé** (*sound*) si chaque fois que $KB \vdash_i \alpha$, il est également vrai que $KB \models \alpha$
- **complet** si chaque fois que $KB \models \alpha$, il est également vrai que $KB \vdash_i \alpha$

Dans ce qui suit, on verra des algorithmes censés et complets pour certaines formes de logique, parfois restreinte à un certain type de phrases.

Logique propositionnelle

Logique propositionnelle

Logique propositionnelle = la logique la plus simple — illustre les idées de base.

- syntaxe
- sémantique
- implications et inférence

Logique propositionnelle : Syntaxe

Grammaire BNF (Backus-Naur Form, voir cours *Compilation*)

Grammaire

$$\begin{aligned}
 \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
 \textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots \\
 \textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \mid [\textit{Sentence}] \\
 &\mid \neg \textit{Sentence} \\
 &\mid \textit{Sentence} \wedge \textit{Sentence} \\
 &\mid \textit{Sentence} \vee \textit{Sentence} \\
 &\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
 &\mid \textit{Sentence} \Leftrightarrow \textit{Sentence}
 \end{aligned}$$

Précédence (décrois.) : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Logique propositionnelle : Syntaxe

- Un symbole de proposition (comme P , Q , $W_{1,3}$, $North$, etc.) est une **phrase atomique**. Chacun de ces symboles est une proposition qui peut être soit **vraie**, soit **fausse**.
- Les symboles **true** et **false** sont deux symboles de proposition fixés.
- un **littéral** est soit une phrase atomique (littéral positif, comme P) soit la négation d'une phrase atomique (littéral négatif, comme $\neg P$).
- La grammaire rappelée précédemment permet de construire des phrases plus complexes en combinant des littéraux.

Logique propositionnelle : Syntaxe

- (conjonction — et) $S_1 \wedge S_2$ ($\wedge$ ressemble à “A” pour “And”)
- (disjonction — ou) $S_1 \vee S_2$
- (implication — implique) $S_1 \Rightarrow S_2$ (où S_1 est le prémisses et S_2 est la conclusion)
- (bicondition — si et seulement si) $S_1 \Leftrightarrow S_2$

Logique propositionnelle : Sémantique

La sémantique définit les règles pour déterminer la **vérité** d'une phrase par rapport à un modèle donné.

En logique propositionnelle, un **modèle** fixe les valeurs de vérité (*true* / *false*) pour chaque symbole de proposition.

Exemple : $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
true *true* *false*

Wumpus : “Dans ce modèle, il y a un trou en (1,2) et en (2,2) mais pas en (3,1).”

(Avec ces symboles, $2^3 = 8$ modèles possibles pouvant être énumérés automatiquement)

Logique propositionnelle : Sémantique

Règles récursives pour évaluer la vérité par rapport à un modèle m :

$\neg S$	est vrai ssi	S est faux	
$S_1 \wedge S_2$	est vrai ssi	S_1 est vrai et	S_2 est vrai
$S_1 \vee S_2$	est vrai ssi	S_1 est vrai ou	S_2 est vrai
$S_1 \Rightarrow S_2$	est vrai ssi	S_1 est faux ou	S_2 est vrai
i.e.,	est faux ssi	S_1 est vrai et	S_2 est faux
$S_1 \Leftrightarrow S_2$	est vrai ssi	$S_1 \Rightarrow S_2$ est vrai et	$S_2 \Rightarrow S_1$ est vrai

Exemple : Si $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
true *true* *false*

Alors, $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{false} \wedge (\text{true} \vee \text{false}) = \text{false} \wedge \text{true} = \text{false}$

Tables de vérité

Les règles peuvent également être exprimées sous la forme de tables de vérité.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Tables de vérité

Remarque concernant $\vee$

P	Q	$P \vee Q$
<i>false</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>true</i>
<i>true</i>	<i>true</i>	<i>true</i>

Les tables de vérité pour $\neg$, $\wedge$, $\vee$ et $\Leftrightarrow$ sont intuitives.

Point de confusion possible : $P \vee Q$ est vrai quand

- P est vrai ;
- ou Q est vrai ;
- ou **les deux**.

Autre opérateur $\exists$: **ou exclusif** (xor) donne *false* quand P et Q sont *true*.

Tables de vérité

Comment comprendre

" $P \Rightarrow Q$ " ?

P	Q	$P \Rightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>true</i>

La table de vérité pour $\Rightarrow$ est moins intuitive. La logique propositionnelle n'impose **pas de lien de causalité** ou de **rapport** entre P et Q .

- "5 est impair implique que Tokyo est la capitale du Japon" : *true*
- "5 est pair implique que Sam est intelligent" : *true* (quel que soit le QI de Sam)

Tables de vérité

Comment comprendre

$P \Rightarrow Q$?

P	Q	$P \Rightarrow Q$
false	false	true
false	true	true
true	false	false
true	true	true

La table de vérité pour $\Rightarrow$ est moins intuitive. La logique propositionnelle n'impose **pas de lien de causalité** ou de **rapport** entre P et Q .

- “5 est impair implique que Tokyo est la capitale du Japon” : **true**
- “5 est pair implique que Sam est intelligent” : **true** (quel que soit le QI de Sam)

Intuition : $P \Rightarrow Q \simeq$ “Si P est vrai, alors je déclare que Q est vrai. Sinon, je ne fais aucune déclaration.”

Le seul cas où cette phrase est fausse : si P est vrai mais Q est faux.

Le monde de Wumpus en phrases

Grâce à la logique propositionnelle, on peut construire une base de connaissances (KB) pour le monde de Wumpus.

Commençons par les aspects **immuables** de ce monde. Pour chaque pièce $[x, y]$, on définit les symboles de proposition suivants :

- $P_{x,y}$: vrai s'il y a un trou en $[x, y]$
- $W_{x,y}$: vrai si Wumpus est en $[x, y]$, vivant ou mort
- $B_{x,y}$: vrai si l'agent perçoit *Breeze* en $[x, y]$
- $S_{x,y}$: vrai si l'agent perçoit *Stench* en $[x, y]$

Le monde de Wumpus en phrases

Exemples de phrases :

- “Il n’y a pas de trou en $[1,1]$ ” :

$$R_1 : \quad \neg P_{1,1}$$

- “Il y a un courant d’air dans une pièce si et seulement s’il y a un trou dans une pièce voisine”. Ceci doit être énoncé pour chaque pièce. Exemples pour les 2 premières pièces :

$$\begin{aligned} R_2 : \quad & B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \\ R_3 : \quad & B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}) \end{aligned}$$

Le monde de Wumpus en phrases

Exemples de phrases :

- “Il n’y a pas de trou en $[1,1]$ ” :

$$R_1 : \quad \neg P_{1,1}$$

- “Il y a un courant d’air dans une pièce si et seulement s’il y a un trou dans une pièce voisine”. Ceci doit être énoncé pour chaque pièce. Exemples pour les 2 premières pièces :

$$R_2 : \quad B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : \quad B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

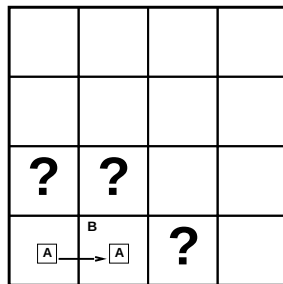
Ces 3 phrases font parties des “règles du jeu”.

Le monde de Wumpus en phrases

En ajoutant les perceptions des 2 premières pièces :

$$R_4 : \quad \neg B_{1,1}$$

$$R_5 : \quad B_{2,1}$$



Inférence : comment en déduire automatiquement que “ $KB \models \neg P_{1,2}$ ” ?

Tables de vérité et inférence

Comment déduire que “ $KB \models \neg P_{1,2}$ ” ?

Première approche : énumération des modèles et inférence par modèle checking.

- Il y a 7 symboles de proposition utiles pour notre problème : $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, $P_{3,1}$
- Cela fait $2^7 = 128$ modèles possibles que l'on peut énumérer
- KB est constituée des phrases R_1 à R_5
- Par énumération, il y a seulement 3 modèles pour lesquels KB est vrai (i.e., les règles R_1 à R_5 sont toutes vraies)

Tables de vérité et inférence

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	false	true	true	true	true	true	true	true	true
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

- si KB est vrai pour une ligne, vérifier que α le soit aussi
- par exemple, si $\alpha = \neg P_{1,2}$, on a bien $KB \models \alpha$ par vérification de modèles (model checking). Ce n'est pas le cas de $\alpha = \neg P_{2,2}$

Inférence par énumération

function TT-ENTAILS ?(*KB*, α) **returns** *true* or *false*

inputs: *KB*, the knowledge base, a sentence in propositional logic

α , the query, a sentence in propositional logic

symbols $\leftarrow$ a list of the proposition symbols in *KB* and α

return TT-CHECK-ALL(*KB*, α , *symbols*, { })

function TT-CHECK-ALL(*KB*, α , *symbols*, *model*) **returns** *true* or *false*

if EMPTY ?(*symbols*) **then**

if PL-TRUE ?(*KB*, *model*) **then return** PL-TRUE ?(α , *model*)

else return *true* # when KB is false, always return true

else do

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return TT-CHECK-ALL(*KB*, α , *rest*, *model* $\cup$ { *P* = *true* }) **and**

 TT-CHECK-ALL(*KB*, α , *rest*, *model* $\cup$ { *P* = *false* })

(TT = Truth Tables)

Inférence par énumération

- énumération (réursive, en profondeur d'abord) de tous les modèles
- PL-TRUE ? retourne vrai si une phrase est satisfaite pour un modèle (PL = Propositional Logic)
- ne retournera vrai que si α est satisfaite pour tous les modèles dans lesquels KB est vrai

Propriétés de TT-ENTAILS ?

Cet algorithme d'inférence est :

- **sensé** car il implémente directement la définition d'implication ($\models$)
 - **complet** car il fonctionne pour n'importe quelles KB et α et se termine toujours (le nombre de modèles étant fini)
 - de **complexité exponentielle** : $O(2^n)$ où n est le nombre de symboles de proposition
- Mauvaise nouvelle** : idem pour tout algorithme d'inférence complet connu en logique propositionnelle

Règles d'inférence et preuves de théorèmes

Règles d'inférence et preuves de théorèmes

La vérification de modèles ([model checking](#), par énumération) permet de déterminer l'implication logique.

La preuve de théorèmes ([theorem proving](#)) est une autre technique pour déterminer l'implication qui utilise l'[équivalence logique](#).

Equivalence logique

Deux phrases sont **équivalentes** ssi elles sont vraies dans les mêmes modèles :

$$\alpha \equiv \beta \text{ si et seulement si } \alpha \models \beta \text{ et } \beta \models \alpha$$

Equivalences logiques classiques (1/2)

$$\begin{aligned} (\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativité de } \wedge \\ (\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativité de } \vee \\ ((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativité de } \wedge \\ ((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativité de } \vee \\ \neg(\neg\alpha) &\equiv \alpha && \text{élimination de la double négation} \end{aligned}$$

Equivalence logique

Equivalences logiques classiques (2/2)

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{élimination de l'implication}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{élimination de } \Leftrightarrow$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{loi de De Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{loi de De Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivité de } \wedge \text{ pour } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivité de } \vee \text{ pour } \wedge$$

Validité et satisfaisabilité

Une phrase est **valide** si elle est vraie dans **tous** les modèles. Une phrase valide est également appelée une **tautologie**.

Exemples

True, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

Validité et satisfaisabilité

Une phrase est **valide** si elle est vraie dans **tous** les modèles. Une phrase valide est également appelée une **tautologie**.

Exemples

True, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

La notion de validité est liée à l'inférence via le **Théorème de Dédution** (déjà connu des grecs anciens).

Théorème (Théorème de Dédution)

$KB \models \alpha$ si et seulement si $(KB \Rightarrow \alpha)$ est valide.

Validité et satisfaisabilité

Une phrase est **satisfaisable** si elle est vraie dans **certains** modèles.

Exemples

$A \vee B$, C

Une phrase est **insatisfaisable** si elle n'est vraie dans **aucun** modèle.

Exemple

$A \wedge \neg A$

Validité et satisfaisabilité

Une phrase est **satisfaisable** si elle est vraie dans **certains** modèles.

Exemples

$A \vee B$, C

Une phrase est **insatisfaisable** si elle n'est vraie dans **aucun** modèle.

Exemple

$A \wedge \neg A$

La notion de satisfaisabilité est liée à l'inférence par le théorème suivant.

Théorème

$KB \models \alpha$ si et seulement si $(KB \wedge \neg \alpha)$ est insatisfaisable.

(= preuve d' α par l'absurde — *reductio ad absurdum*)

Règles d'inférence et preuve de théorèmes

Une **règle d'inférence** est une règle syntaxique qui retourne une conclusion (= une phrase) à partir de prémisses (= ensemble de phrases).

Règles d'inférence et preuve de théorèmes

Une **règle d'inférence** est une règle syntaxique qui retourne une conclusion (= une phrase) à partir de prémisses (= ensemble de phrases).

Notation : pour décrire une règle d'inférence, on la note sous la forme d'une fraction où le numérateur est une liste de prémisses $(p_1, p_2, \dots, p_k)$ et le dénominateur est la conclusion (q) .

$$\frac{p_1, p_2, \dots, p_k}{q}$$

Règles d'inférence et preuve de théorèmes

Une **règle d'inférence** est une règle syntaxique qui retourne une conclusion (= une phrase) à partir de prémisses (= ensemble de phrases).

Notation : pour décrire une règle d'inférence, on la note sous la forme d'une fraction où le numérateur est une liste de prémisses $(p_1, p_2, \dots, p_k)$ et le dénominateur est la conclusion (q) .

$$\frac{p_1, p_2, \dots, p_k}{q}$$

Des règles d'inférence peuvent être appliquées successivement pour dériver une **preuve** : chaîne de conclusions qui mène au but désiré.

Règles d'inférence et preuve de théorèmes

A partir de “ A et B ”, on peut en déduire “Donc A ” (valable aussi pour “ B ” par symétrie)

Règle d'inférence (And-Elimination)

$$\frac{\alpha \wedge \beta}{\alpha}$$

Règles d'inférence et preuve de théorèmes

A partir de “A et B”, on peut en déduire “Donc A” (valable aussi pour “B” par symétrie)

Règle d'inférence (And-Elimination)

$$\frac{\alpha \wedge \beta}{\alpha}$$

Exemple

A partir de “(*WumpusAlive* $\wedge$ *WumpusAhead*)”, on peut inférer “*WumpusAlive*”.

Règles d'inférence et preuve de théorèmes

Une des règles les plus connues : la règle **Modus Ponens** (du verbe latin “ponere”, qui signifie poser, affirmer).

A partir de “Si A alors B ” et “Or, A ” on peut en déduire “Donc B ”

Règle d'inférence (Modus Ponens)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Règles d'inférence et preuve de théorèmes

Une des règles les plus connues : la règle **Modus Ponens** (du verbe latin “ponere”, qui signifie poser, affirmer).

A partir de “Si A alors B ” et “Or, A ” on peut en déduire “Donc B ”

Règle d'inférence (Modus Ponens)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Exemple

A partir de “($WumpusAlive \wedge WumpusAhead$) $\Rightarrow$ $Shoot$ ” et “($WumpusAlive \wedge WumpusAhead$)”, on peut inférer “ $Shoot$ ”.

Règles d'inférence et preuve de théorèmes

Toute **équivalence logique** peut être utilisée comme règle d'inférence.

Exemple

L'équivalence pour l'élimination de $\Leftrightarrow$ donne les 2 règles suivantes :

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)},$$

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}.$$

Méthodes de preuve

- Application de règles d'inférences

- ▶ Génération légitime (sensée) de nouvelles phrases à partir d'anciennes
- ▶ **Preuve** = une séquence d'applications de règles d'inférence sur KB pour prouver α
- ▶ Typiquement : les phrases doivent être traduites dans une **forme normale** pour facilement appliquer les règles d'inférence

⇒ chercher une **preuve** est donc une alternative à l'énumération de modèle : peut être beaucoup plus efficace car ignore les propositions qui n'ont pas de rapport avec l'objectif. Mais le fait d'imposer une forme normale **restreint** la syntaxe et donc l'expressivité.

Formes restreintes de phrases : clauses définies

Une **clause** : disjonction ($\vee$) de littéraux

- un seul littéral positif (un fait)

$$q$$

- au moins un littéral négatif

$$\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_k \vee q$$

Toute clause définie du deuxième type peut être réécrite comme suit (preuve : exercice) :

$$(\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_k \vee q) \equiv (p_1 \wedge p_2 \wedge \dots \wedge p_k) \Rightarrow q$$

Formes restreintes de phrases : clauses définies

Une **clause définie** : disjonction ($\vee$) de littéraux dont **exactement** un est positif. Deux possibilités :

- **un** seul littéral positif (un **fait**)

$$q$$

- **au moins un** littéral négatif

$$\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_k \vee q$$

Toute clause définie du deuxième type peut être réécrite comme suit (preuve : exercice) :

$$(\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_k \vee q) \equiv (p_1 \wedge p_2 \wedge \dots \wedge p_k) \Rightarrow q$$

Formes restreintes de phrases : clauses définies

KB = conjonction ($\wedge$) de clauses définies

Une telle KB est intéressante pour 3 raisons :

- beaucoup de choses peuvent être exprimées sous la forme d'une clause de type

$$(p_1 \wedge p_2 \wedge \dots \wedge p_k) \Rightarrow q$$

Exemple

$$(L_{1,1} \wedge \textit{Breeze}) \Rightarrow B_{1,1}$$

“Si l'agent est en $L_{1,1}$ et qu'il perçoit un courant d'air, alors $B_{1,1}$ ”.

Formes restreintes de phrases : clauses définies

- Avec les clauses définies, on peut utiliser le chaînage avant (**forward chaining**) ou arrière (**backward chaining**) pour inférer grâce à la règle Modus Ponens (voir ci-après). Ce sont 2 algorithmes très naturels et à la base de la **programmation logique**¹.
- **Bonne nouvelle** : Ces algorithmes auront une complexité en **temps linéaire** (en la taille de *KB*).

1. Par exemple, le langage PROLOG est basé sur les clauses définies.

Chaînage avant et arrière (Forward and backward chaining)

Rappel : une KB avec clauses définies ne contient que des faits et des clauses de type

$$(p_1 \wedge p_2 \wedge \dots \wedge p_k) \Rightarrow q$$

Règle d'inférence (Modus Ponens pour KBs avec des clauses définies)

$$\frac{\alpha_1, \dots, \alpha_n, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

Chaînage avant (Forward chaining)

But : PL-FC-ENTAILS ?(KB, q) détermine si un symbole q (*query*) est dérivé d'une implication de KB contenant uniquement des **clauses définies**.

Idée :

- démarre avec les “**faits connus**” dans KB (i.e., les clauses définies constituées d'un seul littéral positif)
- si tous les prémisses (= faits) d'une **implication** de KB sont connus, sa conclusion est ajoutée dans l'ensemble des faits connus
- le processus continue jusqu'à ce que q soit ajouté ou lorsque plus aucune inférence ne peut être réalisée

Illustration du Forward Chaining

Illustration du FC via un graphe particulier (appelé “And-Or Graph”).

Query = Q; KB :

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

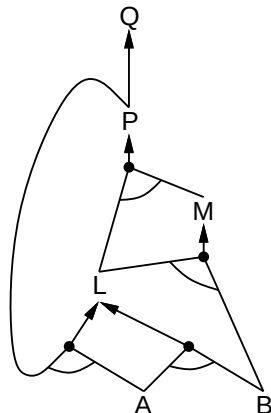


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

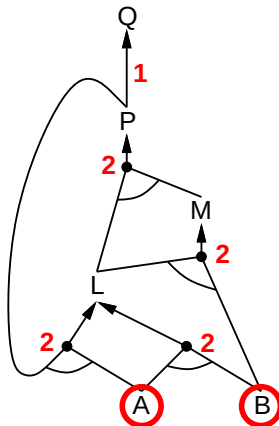


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

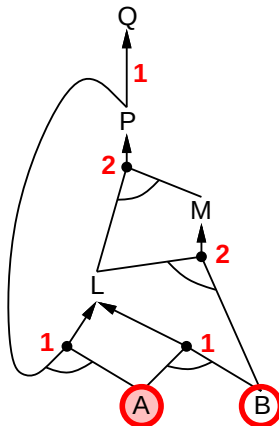


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

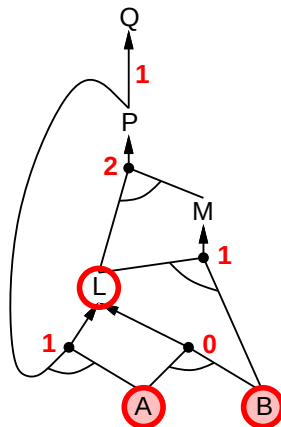


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

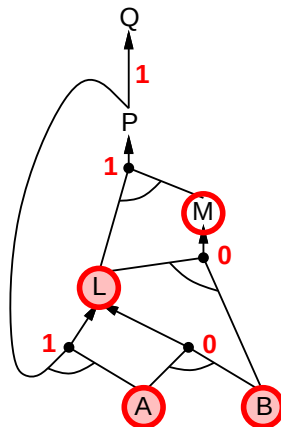


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

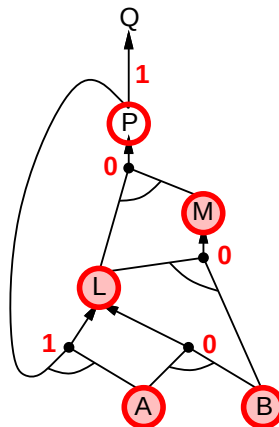


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

$$A$$

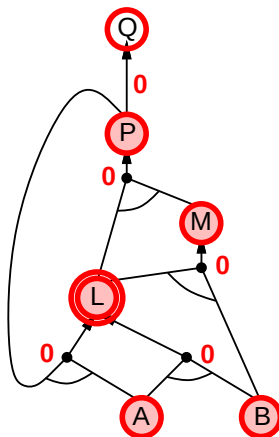
$$B$$


Illustration du Forward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

$$A$$

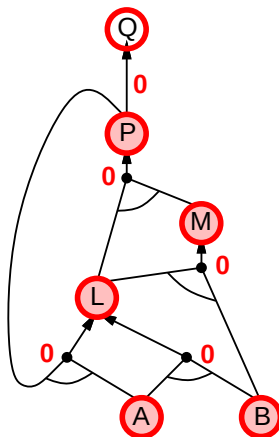
$$B$$


Illustration du Forward Chaining

$$P \Rightarrow Q$$

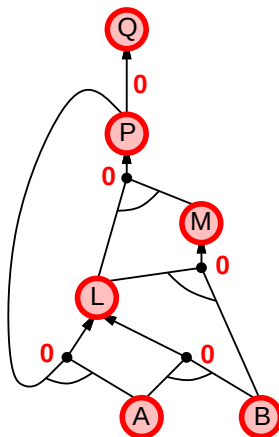
$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

$$A$$

$$B$$


Chaînage avant (Forward chaining)

L'algorithme FC utilise 3 variables locales :

- *agenda* : file de symboles, initialisée avec les symboles connus comme vrai dans *KB*. Contient les faits connus qui n'ont pas encore été "traités".
- *count* : table indexée par les clauses et dont les valeurs sont initialisées avec le nombre de prémisses de chaque clause. Sert à connaître le nombre de prémisses de chaque clause qui ne sont pas encore des faits connus. Si un compteur atteint zéro : tous les prémisses sont connus, et la conclusion peut être ajoutée à l'agenda.
- *inferred* : table indexée par les symboles et dont les valeurs sont initialisées à *false*. Sert à savoir quels symboles ont déjà été traités, pour ne plus les mettre à l'agenda.

Chaînage avant (Forward chaining)

L'algorithme FC utilise 3 variables locales :

- *agenda* : file de symboles, initialisée avec les symboles connus comme vrai dans *KB*. Contient les faits connus qui n'ont pas encore été "traités".
- *count* : table indexée par les clauses et dont les valeurs sont initialisées avec le nombre de prémisses de chaque clause. Sert à connaître le nombre de prémisses de chaque clause qui ne sont pas encore des faits connus. Si un compteur atteint zéro : tous les prémisses sont connus, et la conclusion peut être ajoutée à l'agenda.
- *inferred* : table indexée par les symboles et dont les valeurs sont initialisées à *false*. Sert à savoir quels symboles ont déjà été traités, pour ne plus les mettre à l'agenda.

Chaînage avant (Forward chaining)

L'algorithme FC utilise 3 variables locales :

- *agenda* : file de symboles, initialisée avec les symboles connus comme vrai dans *KB*. Contient les faits connus qui n'ont pas encore été "traités".
- *count* : table indexée par les clauses et dont les valeurs sont initialisées avec le nombre de prémisses de chaque clause. Sert à connaître le nombre de prémisses de chaque clause qui ne sont pas encore des faits connus. Si un compteur atteint zéro : tous les prémisses sont connus, et la conclusion peut être ajoutée à l'agenda.
- *inferred* : table indexée par les symboles et dont les valeurs sont initialisées à *false*. Sert à savoir quels symboles ont déjà été traités, pour ne plus les mettre à l'agenda.

Algorithme Forward Chaining

```

function PL-FC-ENTAILS ?(KB, q) returns true or false
  inputs: KB, the knowledge base – set of propositional definite clauses
           q, the query, a proposition symbol
  local variables: count, inferred, agenda, properly initialized

  while agenda is not empty do
    p ← POP(agenda)
    if p = q then return true
    if inferred [p] = false then
      inferred [p] ← true
      for each clause c in KB where p is in c.PREMISES do
        decrement count [c]
        if count [c] = 0 then
          INSERT(agenda, c.CONCLUSION)
  return false

```

Propriétés du FC

- **Temps ??** linéaire (en la taille de KB) avec une implémentation efficace (file avec opérations en $O(1)$, etc.)
- **Sensé ??** oui car chaque inférence est une application de la règle de Modus Ponens
- **Complet ??** oui (voir preuve ci-après)

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ Preuve : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans **tout** modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ Preuve : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans **tout** modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ **Preuve** : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans **tout** modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ **Preuve** : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans tout modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ **Preuve** : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans tout modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Preuve de complétude

$KB \models q$ pour toute phrase atomique (littéral positif) q

- FC atteint un **point fixé** où plus aucun fait n'est dérivé
- Considérons l'état final comme un modèle m qui assigne *true* / *false* aux symboles (= valeurs de la table *inferred* au point fixé)
- Chaque clause de la KB originale est vraie dans m
 - ▶ **Preuve** : Supposons $a_1 \wedge \dots \wedge a_k \Rightarrow b$ faux dans m
 - ▶ Alors $a_1 \wedge \dots \wedge a_k$ est vraie dans m et b est fausse dans m
 - ▶ En conséquence, l'algorithme n'a pas atteint un point fixé !
- Donc, m est un modèle de KB
- Donc, si $KB \models q$, alors q est vrai dans **tout** modèle de KB , incluant m : il y aurait une contradiction si l'algorithme n'impliquait pas q

Utilisation du Forward Chaining

En pratique, comment notre agent logique peut-il utiliser le FC ?
Approche dirigée par les données (*data-driven*).

- On démarre avec les faits connus
- Ils peuvent être utilisés pour dériver des conclusions à partir des perceptions, souvent sans requête (*query*) particulière à l'esprit
- Par exemple l'agent Wumpus peut dire (TELL) ses perceptions à *KB* en utilisant un algorithme FC incrémental (de nouveaux faits peuvent être ajoutés à l'agenda pour initier de nouvelles inférences)
- A partir des faits connus (initialement ou inférés), l'agent prend la meilleure décision. Celle-ci peut donc changer en fonction des perceptions.

Chaînage arrière (Backward chaining — BC)

Idée : travailler en arrière depuis la requête q (approche dirigée par l'objectif)

Preuve de q par BC :

- si q est connu, retourner vrai
- sinon, pour chaque implication i ayant comme conclusion q
 - ▶ si tous les prémisses de i sont vrais (preuve par BC), alors retourner vrai
- retourner faux

Chaînage arrière (Backward chaining — BC)

Idée : travailler en arrière depuis la requête q (approche dirigée par l'objectif)

Preuve de q par BC :

- si q est connu, retourner vrai
- sinon, pour chaque implication i ayant comme conclusion q
 - ▶ si tous les prémisses de i sont vrais (preuve par BC), alors retourner vrai
- retourner faux

Remarques :

- Pour éviter les boucles : vérifier si un nouveau sous-objectif à prouver est déjà dans la pile d'objectifs
- Pour éviter le travail inutile : vérifier si un nouveau sous-objectif
 - ▶ a déjà été prouvé (vrai)
 - ▶ a déjà été infirmé

⇒ avec implémentation efficace : temps linéaire également

Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

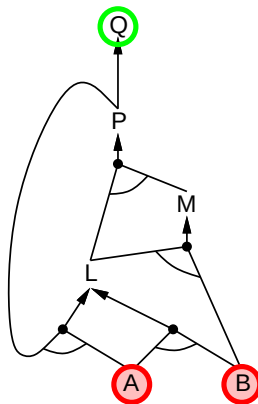


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

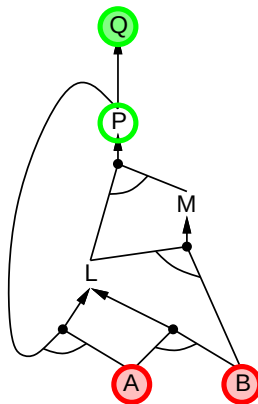


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

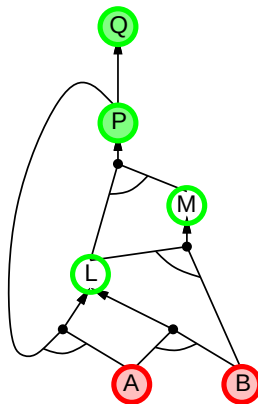


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

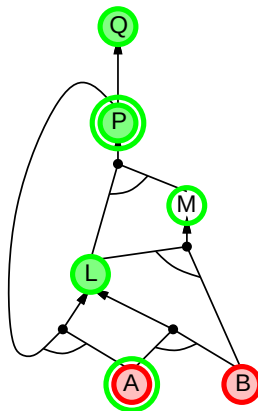


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

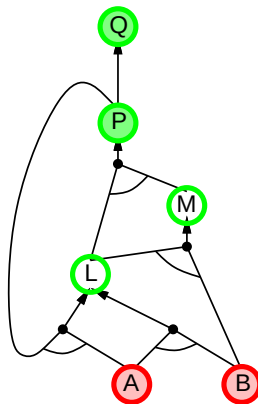


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

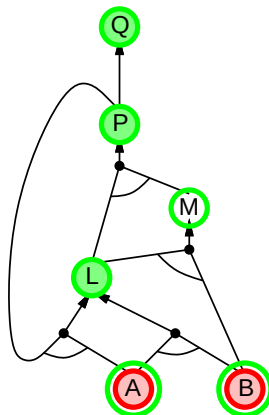


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

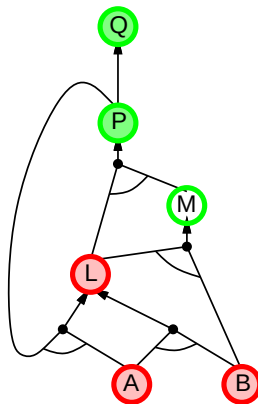


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

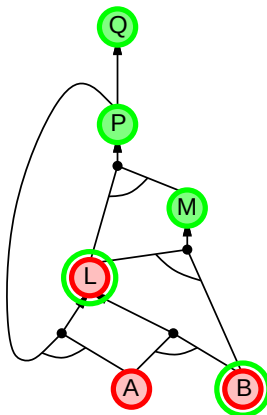


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

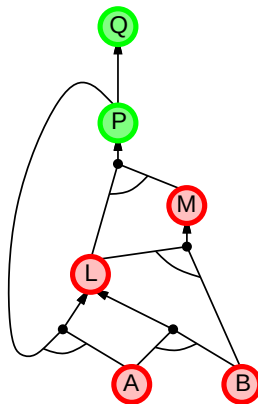


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

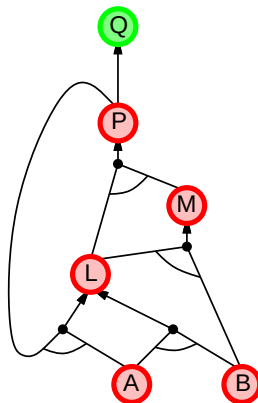


Illustration du Backward Chaining

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

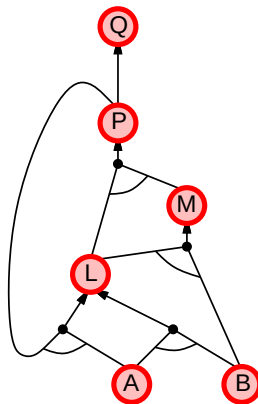
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward vs. backward chaining

FC est dirigé par les **données** : processus automatique, “inconscient”, e.g., reconnaissance d'objets, routines de décisions

Peut réaliser beaucoup de travail sans rapport avec l'objectif.

BC est dirigé par l'**objectif**, approprié pour la résolution de problèmes

En pratique, le nombre d'itérations du BC peut être **bien inférieur** à celui du FC.

Mais ces 2 algorithmes utilisent une forme restreinte (clauses définies) pour exprimer la connaissance.

Preuve par résolution

La **preuve par résolution** est

- un autre algorithme d'inférence
- qui utilise une forme normale **universelle** pour la logique propositionnelle (i.e., toutes les phrases peuvent être réécrites sous cette forme)
- utilise une règle d'inférence unique (la **résolution**) qui est **sensée** et **complète** pour la logique propositionnelle².
- basée sur le principe de preuve par contradiction

2. Non prouvé dans ce cours (mais bien dans le Chap. 7 du livre de référence).

Forme Normale Conjonctive

Forme Normale Conjonctive (Conjunctive Normal Form — CNF) :
 conjonction ($\wedge$) de $\underbrace{\text{disjonctions } (\vee) \text{ de littéraux}}_{\text{clauses}}$

Exemple

$$(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$$

Règle de résolution pour CNF

La règle (pour CNF) suivante est sensée et complète pour la logique propositionnelle. Elle permet de **résoudre** 2 clauses possédant des littéraux complémentaires.

Règle d'inférence (Resolution)

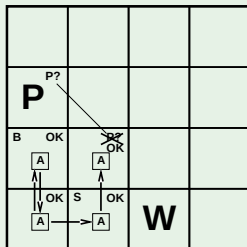
$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

où ℓ_i et m_j sont des littéraux **complémentaires**.

Remarque : le complémentaire de q est $\neg q$ et vice-versa.

Règle de résolution pour CNF

Exemple



$$\frac{P_{1,3} \vee P_{2,2}, \quad \neg P_{2,2}}{P_{1,3}}$$

Règle de résolution pour CNF

Exemple

Une paire de clauses peut donner plusieurs **résolvants** :

$$\frac{\neg P_{2,1} \vee B_{1,1}, \quad \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}}{\neg B_{1,1} \vee P_{1,2} \vee B_{1,1}},$$

$$\frac{\neg P_{2,1} \vee B_{1,1}, \quad \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}}{P_{1,2} \vee P_{2,1} \vee \neg P_{2,1}}.$$

Conversion en CNF

Grâce aux équivalences logiques, toute phrase de logique propositionnelle peut être convertie en CNF.

- Elimination de $\Leftrightarrow$: remplacer $\alpha \Leftrightarrow \beta$ par $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
- Elimination de $\Rightarrow$: remplacer $\alpha \Rightarrow \beta$ par $\neg\alpha \vee \beta$.

Conversion en CNF

Grâce aux équivalences logiques, toute phrase de logique propositionnelle peut être convertie en CNF.

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

- Elimination de $\Leftrightarrow$: remplacer $\alpha \Leftrightarrow \beta$ par $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
- Elimination de $\Rightarrow$: remplacer $\alpha \Rightarrow \beta$ par $\neg\alpha \vee \beta$.

Conversion en CNF

Grâce aux équivalences logiques, toute phrase de logique propositionnelle peut être convertie en CNF.

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

- Elimination de $\Leftrightarrow$: remplacer $\alpha \Leftrightarrow \beta$ par $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

- Elimination de $\Rightarrow$: remplacer $\alpha \Rightarrow \beta$ par $\neg\alpha \vee \beta$.

Conversion en CNF

Grâce aux équivalences logiques, toute phrase de logique propositionnelle peut être convertie en CNF.

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

- Elimination de $\Leftrightarrow$: remplacer $\alpha \Leftrightarrow \beta$ par $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

- Elimination de $\Rightarrow$: remplacer $\alpha \Rightarrow \beta$ par $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

Conversion en CNF

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

- Déplacer $\neg$ vers l'intérieur grâce à De Morgan et simplifier les doubles négations.
- Appliquer la distributivité (de $\vee$ pour $\wedge$).

Conversion en CNF

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

- Déplacer $\neg$ vers l'intérieur grâce à De Morgan et simplifier les doubles négations.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

- Appliquer la distributivité (de $\vee$ pour $\wedge$).

Conversion en CNF

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

- Déplacer $\neg$ vers l'intérieur grâce à De Morgan et simplifier les doubles négations.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

- Appliquer la distributivité (de $\vee$ pour $\wedge$).

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Il reste une conjonction de 3 clauses.

Algorithme de résolution

Idée : preuve par contradiction, i.e., montrer que $KB \wedge \neg\alpha$ est insatisfaisable

- D'abord, conversion de $KB \wedge \neg\alpha$ en CNF
- Ensuite, appliquer la règle de résolution aux clauses résultantes
- Chaque paire de clauses qui contient 2 littéraux complémentaires est résolue pour produire une nouvelle clause
- Le processus continue jusqu'à ce que l'une des deux choses suivantes se produit :
 - ▶ il n'y a plus de clause qui peut être produite : $KB \not\models \alpha$
 - ▶ deux clauses produisent la clause vide : $KB \models \alpha$

Algorithme de résolution

Idée : preuve par contradiction, i.e., montrer que $KB \wedge \neg\alpha$ est insatisfaisable

- D'abord, conversion de $KB \wedge \neg\alpha$ en CNF
- Ensuite, appliquer la règle de résolution aux clauses résultantes
- Chaque paire de clauses qui contient 2 littéraux complémentaires est résolue pour produire une nouvelle clause
- Le processus continue jusqu'à ce que l'une des deux choses suivantes se produit :
 - ▶ il n'y a plus de clause qui peut être produite : $KB \not\models \alpha$
 - ▶ deux clauses produisent la clause **vide** : $KB \models \alpha$

La clause **vide** est obtenue par la règle de résolution appliquée sur deux littéraux complémentaires $\Rightarrow$ contradiction.

Exemple (Wumpus)

On part de :

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \text{et } \alpha = \neg P_{1,2}$$

Exemple (Wumpus)

On part de :

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \text{et } \alpha = \neg P_{1,2}$$

En CNF, on a vu ci-avant que $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ correspond à 3 clauses :

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Exemple (Wumpus)

On part de :

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \text{et } \alpha = \neg P_{1,2}$$

En CNF, on a vu ci-avant que $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ correspond à 3 clauses :

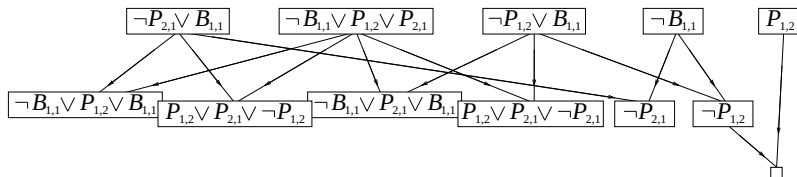
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Donc, $KB \wedge \neg \alpha$ est constitué de 5 clauses en CNF : les 3 clauses ci-dessus, et les 2 suivantes :

$$(\neg B_{1,1}) \wedge (P_{1,2})$$

Exemple (Wumpus)

On applique la règle de résolution à chaque paire de clauses. Dans cet exemple, on trouve la clause vide qui correspond à la contradiction $\neg P_{1,2} \wedge P_{1,2}$.



Algorithme de résolution

```

function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
            $\alpha$ , the query, a sentence in propositional logic

  clauses  $\leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
  new  $\leftarrow \{\}$ 
  loop do
    for each pair of clauses  $C_i, C_j$  in clauses do
      resolvents  $\leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if resolvents contains the empty clause then return true
      new  $\leftarrow$  new  $\cup$  resolvents
    if new  $\subseteq$  clauses then return false
    clauses  $\leftarrow$  clauses  $\cup$  new

```

PL-RESOLVE : retourne l'ensemble des clauses possibles obtenues par la règle de résolution appliquée à C_i et C_j

Résumé du chapitre

Résumé du chapitre

Les agents logiques appliquent l'**inférence** à une **base de connaissances** pour dériver de nouvelles informations et prendre des décisions

Concepts de base de la logique :

- **syntaxe** : structure formelle de **phrases**
- **sémantique** : **vérité** des phrases par rapport à des **modèles**
- **implication** : le fait qu'une phrase soit vraie en implique une autre
- **inférence** : dériver des phrases d'autres phrases
- **sensé** : se dit d'une procédure d'inférence qui ne produit que des phrases obtenues par implication logique
- **complet** : se dit d'une procédure d'inférence qui produit toutes les phrases pouvant être obtenues par implication logique

Résumé du chapitre

Algorithmes d'inférence vus dans ce chapitre :

- chaînage avant et arrière (**Forward**, **backward** chaining) : temps linéaire et complet pour des clauses **définies** (Forme de Horn).
- l'algorithme de **résolution** est complet pour la logique propositionnelle.

La **logique propositionnelle** est une forme de logique très simple mais qui ne peut pas tout exprimer.